

TEDx



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

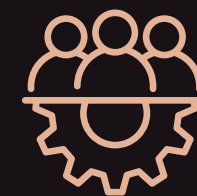


Panoramica



Navigazione tra Talk Suggestiti

- Integrazione e trattamento del dataset **watch_next**
- Modifica del job PySpark per aggiornare il DWH
- Analisi dell'utilizzo del dato
- Funzionalità proposte per gli utenti
(es. suggerimenti personalizzati, raccomandazioni simili)



Nuova Funzionalità sulla Board

- Ideare e implementare una funzionalità nuova
- Approccio data-driven: sfruttare i dati per valore aggiunto
- Obiettivo: strutturare i dati in un modello a grafo

Python

```
# ----- START: Calculate next_watch -----

# 1. Explode tags: Create rows like (_id, tag)
# Ensure 'tags' column exists and handle potential nulls before exploding
if 'tags' in tedx_dataset_agg.columns:
    # Handle talks that might not have tags (due to left join earlier)
    tedx_dataset_agg = tedx_dataset_agg.withColumn("tags", coalesce(col("tags"), array().cast("array<string>")))
    exploded_tags = tedx_dataset_agg.select("_id", explode("tags").alias("tag"))

# 2. Self-join on tag to find talks with common tags
# Alias the exploded dataframe to perform self-join
t1 = exploded_tags.alias("t1")
t2 = exploded_tags.alias("t2")

# Join where tags match AND the talk IDs are different
related_pairs = t1.join(t2, (col("t1.tag") == col("t2.tag")) & (col("t1._id") != col("t2._id")), "inner") \
    .select(col("t1._id").alias("source_id"), col("t2._id").alias("related_id")) \
    .distinct() # Ensure unique pairs (source_id, related_id)

# 3. Aggregate related IDs for each source ID
# Use collect_set to get unique related IDs per source talk
next_watch_mapping = related_pairs.groupBy("source_id") \
    .agg(collect_set("related_id").alias("next_watch")) \
    .withColumnRenamed("source_id", "join_id") # Rename to avoid clash

next_watch_mapping.printSchema()

# 4. Left join the next_watch mapping back to the main aggregated dataset
tedx_final_dataset = tedx_dataset_agg.join(
    next_watch_mapping,
    tedx_dataset_agg["_id"] == next_watch_mapping["join_id"],
    "left"
).drop("join_id") # Drop the join key from the mapping table

# 5. Handle talks with no related items (replace null next_watch with empty list)
# Determine the type of _id for correct casting of empty list lit([])
# Assuming _id is string based on typical CSV reads and usage. Adjust if it's numeric.
id_type = dict(tedx_final_dataset.dtypes)['_id']
if id_type.startswith("array"): # Should not happen for _id, but defensive check
    print(f"Warning: _id column has unexpected type {id_type}")
    # Fallback or error handling needed
elif id_type in ("int", "bigint", "long", "short", "byte"):
    array_type = "array<long>" # Or appropriate numeric type
else: # Default to string
    array_type = "array<string>"

tedx_final_dataset = tedx_final_dataset.withColumn(
    "next_watch",
    coalesce(col("next_watch"), lit(None).cast(array_type)) # Use lit(None).cast for empty array
    # Alternative using array() but needs specific type: coalesce(col("next_watch"), array().cast(array_type))
)
```

```
tedx_final_dataset.printSchema()

# ----- END: Calculate next_watch -----

else:
    print("Warning: 'tags' column not found after aggregation. Skipping next_watch calculation.")
    # Add an empty next_watch column if tags column was missing
    tedx_final_dataset = tedx_dataset_agg.withColumn("next_watch", array().cast("array<string>"))

else:
    print("Error: 'id' column missing in tedx_dataset_main for joining tags. Exiting.")
    sys.exit(1) # Or handle error appropriately

else:
    print("Warning: 'id' or 'tag' column not found in tags_dataset. Skipping tags aggregation and next_watch calculation.")
    # Add empty tags and next_watch columns if tags couldn't be processed
    tedx_final_dataset = tedx_dataset_main.withColumn("tags", array().cast("array<string>")) \
        .withColumn("next_watch", array().cast("array<string>")) \
        .select(col("id").alias("_id"), col("***")) \
        .drop("id") # Ensure _id exists even if tags failed

# --- Writing to MongoDB ---
# Use the final dataset with the 'next_watch' column
if 'tedx_final_dataset' in locals():
    write_mongo_options = {
        "connectionName": "Mongodbatlas connection", # Make sure this connection is configured in Glue
        "database": "unibg_tedx_2025",
        "collection": "tedx_data",
        "ssl": "true",
        "ssl.domain_match": "false"
    }

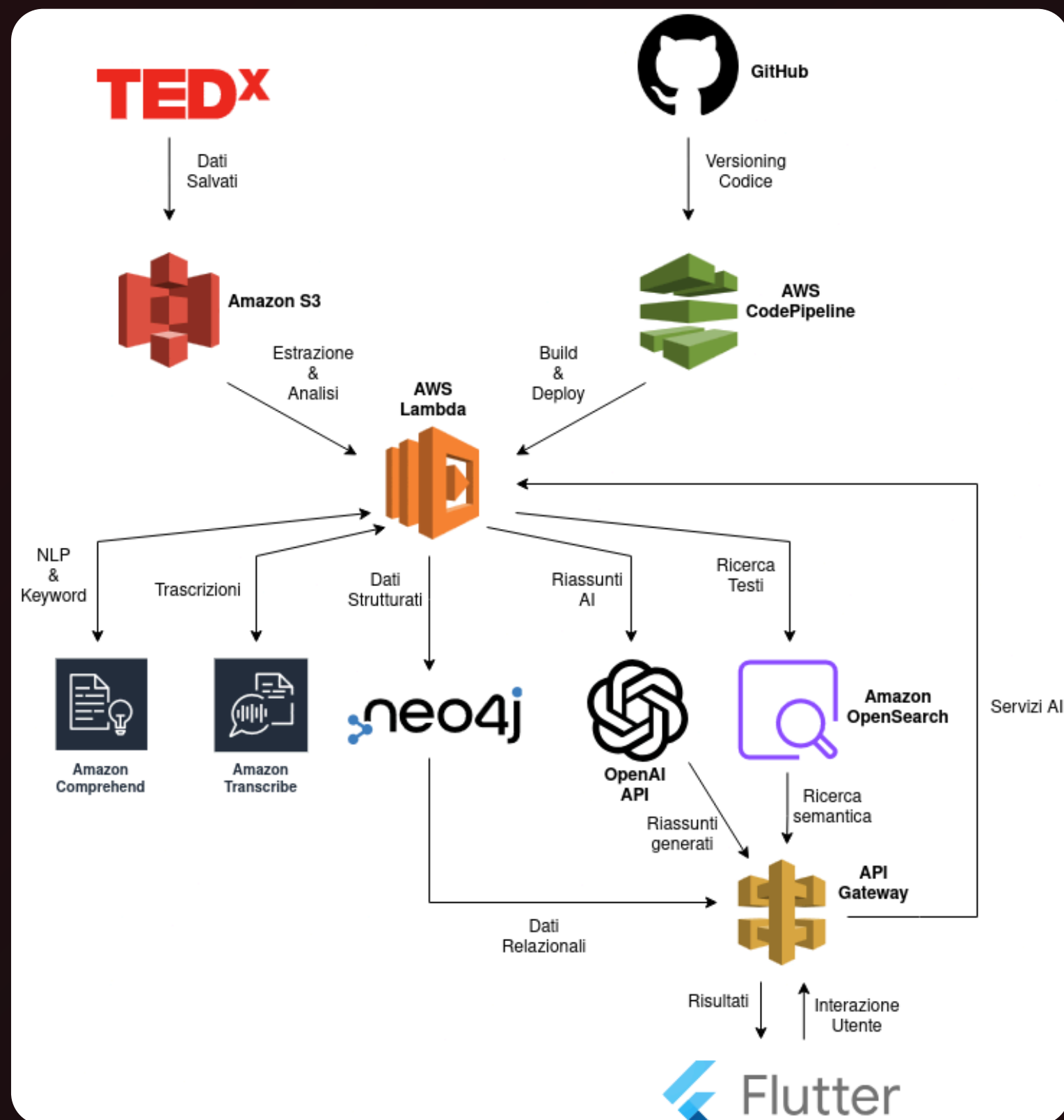
    tedx_dataset_dynamic_frame = DynamicFrame.fromDF(tedx_final_dataset, glueContext, "nested")

    print("Writing final dataset to MongoDB...")
    glueContext.write_dynamic_frame.from_options(
        frame=tedx_dataset_dynamic_frame,
        connection_type="mongodb",
        connection_options=write_mongo_options
    )
    print("Successfully wrote data to MongoDB.")
else:
    print("Error: Final dataset ('tedx_final_dataset') was not created due to previous errors. Skipping MongoDB write.")
```

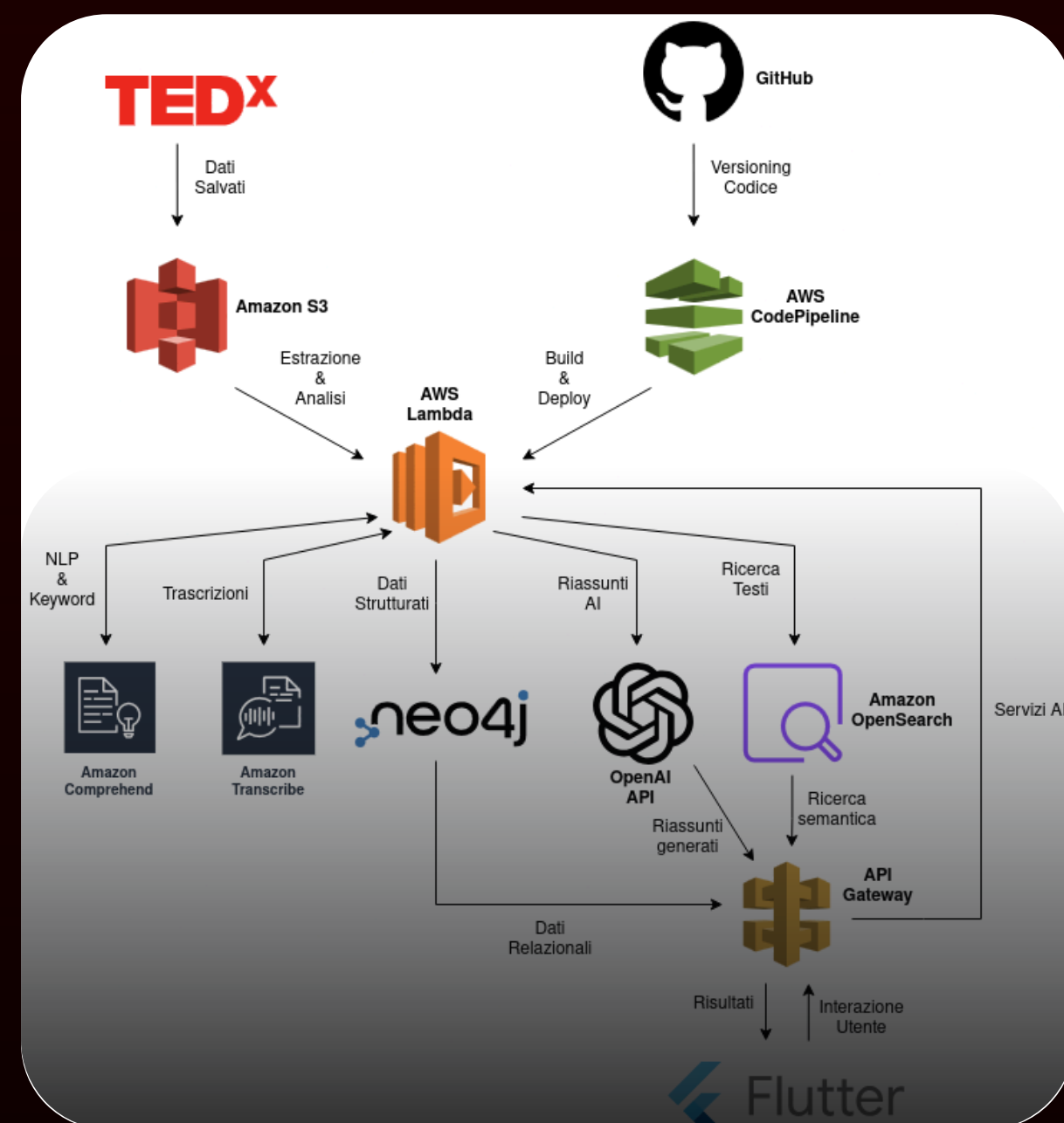
Risultato

```
▶ {
  _id: "567505"
  slug: "ben_proudfoot_the_true_story_of_the_iconic_tagline_because_i_m_worth_i..."
  speakers: "Ben Proudfoot"
  title: "The true story of the iconic tagline "Because I'm worth it." | The Fin..."
  url: "https://www.ted.com/talks/ben_proudfoot_the_true_story_of_the_iconic_t..."
  description: "From two-time Oscar winner Ben Proudfoot comes THE FINAL COPY OF ILON ..."
  duration: "1059"
  publishedAt: "2025-03-07T13:49:56Z"
  tags: Array (8)
  next_watch: Array (2474)
    0: "465069"
    1: "272647"
    2: "155377"
    3: "1078"
    4: "504093"
    5: "81146"
    6: "31246"
    7: "45790"
    8: "481763"
    9: "283454"
    10: "265571"
    11: "115212"
    12: "67584"
    13: "24110"
```

```
▶ {
  _id: "567505"
  slug: "ben_proudfoot_the_true_story_of_the_iconic_tagline_because_i_m..."
  speakers: "Ben Proudfoot"
  title: "The true story of the iconic tagline "Because I'm worth it."
  url: "https://www.ted.com/talks/ben_proudfoot_the_true_story_of_the_f..."
  description: "From two-time Oscar winner Ben Proudfoot comes THE FINA..."
  duration: "1059"
  publishedAt: "2025-03-07T13:49:56Z"
  tags: Array (8)
  next_watch: Array (2474)
```

MEMO



Da MongoDB a Neo4j

Implementazione della pipeline dati che trasferisce i talk TEDx e le loro relazioni da MongoDB a Neo4j, creando nodi e collegamenti nella mappa mentale. Questo consente la visualizzazione dinamica dei contenuti nell'app Flutter.

```
2 import os
3 import json
4 import sys
5 from pymongo import MongoClient, errors as pymongo_errors # Import specifico per errori
6 from neo4j import GraphDatabase, basic_auth, exceptions as neo4j_exceptions # Import specifico per errori
7 from urllib.parse import quote_plus
8 import traceback # Import per stack trace
9
10 # --- Configuration from Environment Variables ---
11 # CREDENZIALI DA AGGIUNGERE A MANO
12 # Queste variabili devono essere impostate nell'ambiente di esecuzione del job Glue
13 # Puoi usare AWS Glue Job Parameters o AWS Secrets Manager per gestire le credenziali in modo sicuro
14
15 # Credenziali da configurare tramite ambiente o secret manager
16 MONGO_USER = "[inserire il proprio username MongoDB]"
17 MONGO_PASSWORD = "[inserire la propria password MongoDB]"
18 MONGO_HOST = "[inserire il proprio host MongoDB, es. cluster0.xxx.mongodb.net]"
19 MONGO_DB_NAME = "[inserire il nome del database MongoDB, es. unibg_tedx_2025]"
20 MONGO_COLLECTION_NAME = "[inserire il nome della collezione MongoDB, es. texd_data]"
21
22 NEO4J_URI = "[inserire l'URI Neo4j, es. neo4j+s://... ]"
23 NEO4J_USER = "[inserire il proprio username Neo4j]"
24 NEO4J_PASSWORD = "[inserire la propria password Neo4j]"
25
26 # --- Basic Validation ---
27 # Aggiungi i nuovi campi Mongo alla validazione
28 if not all([MONGO_USER, MONGO_PASSWORD, MONGO_HOST, MONGO_DB_NAME, MONGO_COLLECTION_NAME,
29            NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD]):
30     print("Error: Missing one or more configuration variables (MONGO_*, NEO4J_*)")
31     # In Glue, esci con un codice di errore per segnalare il fallimento del job
32     sys.exit("Configuration Error: Missing required variables.")
33
34 # --- Global Clients (initialized outside main logic for clarity) ---
35 mongo_client = None
36 neo4j_driver = None
37
38 # --- Client Initialization Function ---
39 Windsurf: Refactor | Explain | ✕
40 def init_clients():
41     """Initializes MongoDB and Neo4j clients if not already initialized."""
42     global mongo_client, neo4j_driver
43
44     # --- MongoDB Initialization ---
45     if mongo_client is None:
46         try:
47             print("Initializing MongoDB client...")
48             encoded_user = quote_plus(MONGO_USER)
49             encoded_password = quote_plus(MONGO_PASSWORD)
50             mongo_uri_string = f"mongodb+srv://{encoded_user}:{encoded_password}@{MONGO_HOST}?retryWrites=true&w=majority"
51             print(f"Connecting to MongoDB host: {MONGO_HOST}")
52             # Aumenta leggermente il timeout per Glue se necessario
53             mongo_client = MongoClient(mongo_uri_string, serverSelectionTimeoutMS=10000)
54             mongo_client.admin.command('ping') # Verifica la connessione
55             print("MongoDB connection successful.")
56         except (pymongo_errors.ConfigurationError, pymongo_errors.ConnectionFailure) as e:
57             print(f"Error connecting to MongoDB: {e}")
58             mongo_client = None
59             raise # Rilancia l'eccezione per fermare l'esecuzione
```

```
60     print(f"An unexpected error occurred during MongoDB initialization: {e}")
61     mongo_client = None
62     raise
63
64 # --- Neo4j Initialization ---
65 if neo4j_driver is None:
66     try:
67         print("Initializing Neo4j driver...")
68         neo4j_driver = GraphDatabase.driver(
69             NEO4J_URI,
70             auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD),
71             connection_timeout=20 # Aumenta leggermente il timeout per Glue se necessario
72         )
73         neo4j_driver.verify_connectivity()
74         print("Neo4j connection successful.")
75     except (neo4j_exceptions.ServiceUnavailable, neo4j_exceptions.AuthError) as e:
76         print(f"Error connecting to Neo4j: {e}")
77         neo4j_driver = None
78         raise # Rilancia l'eccezione per fermare l'esecuzione
79     except Exception as e:
80         print(f"An unexpected error occurred during Neo4j initialization: {e}")
81         neo4j_driver = None
82         raise
83
84 # --- Client Cleanup Function ---
85 Windsurf: Refactor | Explain | ✕
86 def cleanup_clients():
87     """Closes Neo4j driver connection."""
88     global neo4j_driver
89     if neo4j_driver:
90         try:
91             print("Closing Neo4j driver.")
92             neo4j_driver.close()
93             neo4j_driver = None
94         except Exception as e:
95             print(f"Warning: Error closing Neo4j driver: {e}") # Logga l'errore ma non bloccare
```



```

167 # --- Main Execution Logic for Glue Python Shell ---
168 if __name__ == "__main__":
169
170     print("Starting AWS Glue Python Shell Job...")
171     processed_nodes = 0
172     created_relationships = 0
173     clients_initialized_successfully = False # Flag per il blocco finally
174
175     try:
176         # --- Initialize Clients ---
177         print("Attempting to initialize database clients...")
178         init_clients()
179         clients_initialized_successfully = True # Se arriva qui, l'init è andato a buon fine
180         print("Database clients initialized successfully.")
181
182         # --- Get MongoDB Collection ---
183         # Non serve controllare se i client sono None qui, init_clients lancia eccezione se fallisce
184         db = mongo_client[MONGO_DB_NAME]
185         collection = db[MONGO_COLLECTION_NAME]
186         print(f"Accessed MongoDB collection: {MONGO_DB_NAME}.{MONGO_COLLECTION_NAME}")
187
188         # --- Fetch Data ---
189         print("Fetching talks data from MongoDB...")
190         try:
191             # Considera find({}).limit(N) per testare con pochi dati
192             # Considera proiezione se non servono tutti i campi: collection.find({}, {'_id': 1, 'title': 1, 'next_watch': 1, ...})
193             talks_cursor = collection.find({})
194             talks_data = list(talks_cursor) # ATTENZIONE: Carica tutto in memoria! Ok per dati < memoria worker Glue
195             found_count = len(talks_data)
196             print(f"Found {found_count} talks in MongoDB.")
197             if not talks_data:
198                 print("No data found in MongoDB collection. Job will exit successfully.")
199                 # Uscire qui se non c'è nulla da fare è corretto
200             else:
201                 # --- Phase 1: Create/Update Nodes ---
202                 print("Phase 1: Creating/Updating Talk nodes in Neo4j...")
203                 with neo4j_driver.session(database="neo4j") as session: # Specifica il database Neo4j target
204                     for i, talk in enumerate(talks_data):
205                         if '_id' not in talk or talk['_id'] is None:
206                             print(f"Skipping document at index {i} due to missing or null '_id'.")
207                             continue
208                         try:
209                             # Usa execute_write per la gestione automatica delle transazioni e dei retry
210                             session.execute_write(create_or_update_talk_node, talk)
211                             processed_nodes += 1
212                             if processed_nodes % 100 == 0 or processed_nodes == found_count:
213                                 print(f"Processed {processed_nodes}/{found_count} nodes...")
214                         except Exception as e:
215                             # Logga l'errore ma continua con gli altri nodi
216                             print(f"Error processing node for talk ID {talk.get('_id', 'N/A')}: {e}")
217                             traceback.print_exc() # Stampa lo stack trace per debug
218                             # Potresti voler salvare gli ID falliti per un'analisi successiva
219                             continue

```

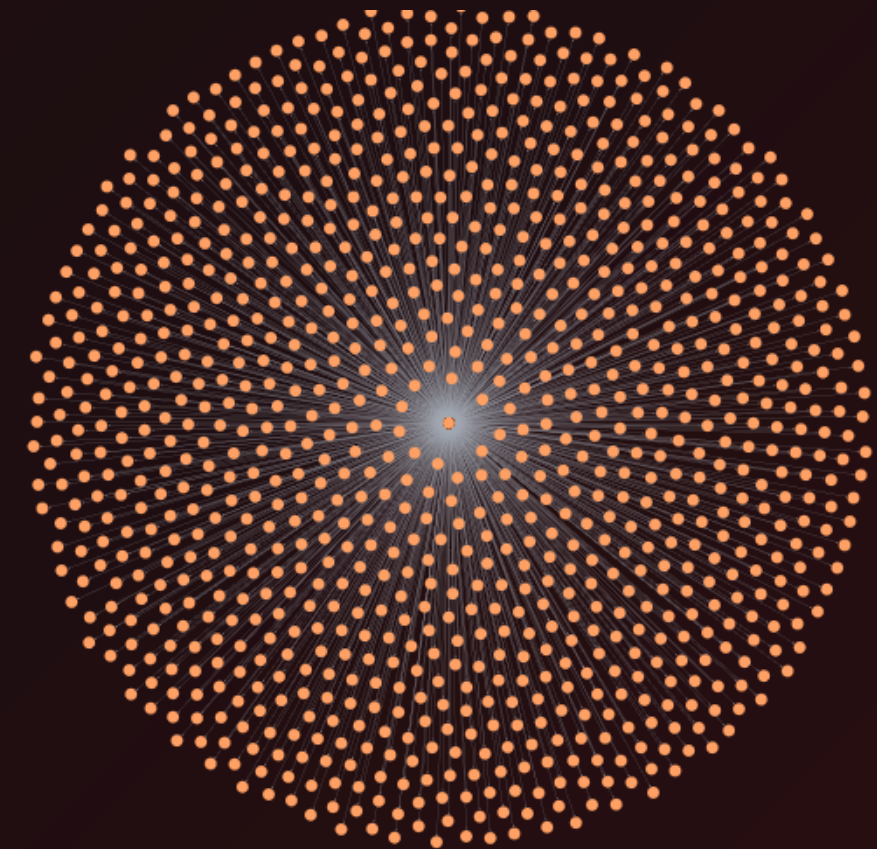
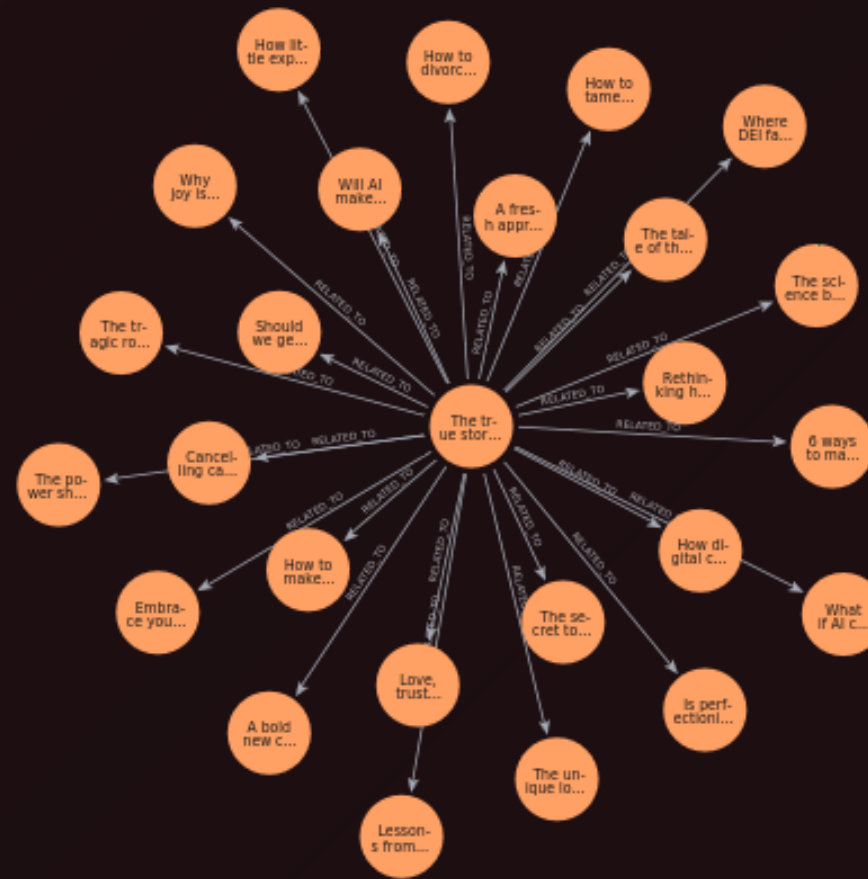
```

222
223 # --- Phase 2: Create Relationships ---
224 print("Phase 2: Creating RELATED_TO relationships in Neo4j...")
225 with neo4j_driver.session(database="neo4j") as session:
226     for i, talk in enumerate(talks_data):
227         source_id = talk.get('_id')
228         next_watch_list = talk.get('next_watch')
229
230         if not source_id:
231             print(f"Skipping relationship creation for talk at index {i} without source ID.")
232             continue
233
234         # Assicurati che 'next_watch' sia una lista e non None/vuota
235         if isinstance(next_watch_list, list) and next_watch_list:
236             for related_id in next_watch_list:
237                 # Salta auto-relazioni e valori None/vuoti nella lista
238                 if related_id is None or str(related_id).strip() == "" or str(source_id) == str(related_id):
239                     if str(source_id) == str(related_id):
240                         # print(f"Skipping self-relationship for talk ID {source_id}") # Log opzionale
241                         pass
242                     else:
243                         print(f"Skipping invalid/null related_id '{related_id}' for source {source_id}")
244                         continue
245
246                 try:
247                     # Usa execute_write anche qui
248                     session.execute_write(create_relationship, source_id, related_id)
249                     created_relationships += 1 # Incrementa qui, anche se MERGE potrebbe non creare nulla di nuovo
250                     if created_relationships % 100 == 0:
251                         print(f"Attempted to create/merge {created_relationships} relationships...")
252                 except Exception as e:
253                     # create_relationship ora gestisce gli errori di query internamente e logga
254                     # Non c'è bisogno di fare molto altro qui, a meno che non si voglia interrompere il job
255                     # Questo punto non dovrebbe essere raggiunto se create_relationship cattura le eccezioni di run()
256                     print(f"Unexpected error during relationship write transaction ({source_id})->({related_id}): {e}")
257                     traceback.print_exc()
258                     continue # Continua con la prossima relazione/talk
259
260     print(f"Finished Phase 2. Attempted to create/merge approximately {created_relationships} relationships (MERGE skips duplicates).")
261     print("Data synchronization process completed.")
262
263 except Exception as e:
264     print(f"Error fetching data from MongoDB or during processing phases: {e}")
265     traceback.print_exc()
266     # Considera di uscire con errore se il fetch fallisce
267     sys.exit("Job failed during data fetching or processing.")
268
269 # --- Gestione Errori Generali (es. Connessione Iniziale) ---
270 except (pymongo_errors.ConnectionFailure, neo4j_exceptions.ServiceUnavailable, neo4j_exceptions.AuthError) as e:
271     print(f"FATAL: Database connection or authentication error during initialization: {e}")
272     traceback.print_exc()
273     # Segnala il fallimento del job a Glue
274     sys.exit("Job failed due to database connection/auth error.")
275 except Exception as e: # Catch generico per altri problemi imprevisti (es. errori di logica nello script)
276     print(f"FATAL: An unexpected error occurred: {e}")
277     traceback.print_exc()
278     # Segnala il fallimento del job a Glue
279     sys.exit("Job failed due to an unexpected error.")

```


Neo4J

↓ SCARICA



Criticità



01. Debugging complesso

Il job non dispone di strumenti intuitivi per il debug. I log risultano frammentati e difficili da seguire in caso di errori.



02. Tempi lunghi di esecuzione

Il processo di compilazione e importazione del codice Python su EC2 per generare il modello a grafo è risultato lento da eseguire, impiegando varie ore



03. Limitazione per il piano free

Il database ha raggiunto il limite di 400.000 relazioni previsto dal tier gratuito di Neo4j, generando un errore che ha impedito l'aggiunta di nuove relazioni e bloccato l'espansione del grafo.



04. Rischio di duplicazione dati

L'assenza di controlli idempotenti può causare la duplicazione dei nodi e delle relazioni in Neo4j a ogni esecuzione del job.



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Possibili Evoluzioni

Miglioramenti al processo ETL e all'integrazione dati per rendere la pipeline più efficiente, scalabile e affidabile nel tempo.

Utilizzo dei piani premium

L'utilizzo dei piani premium di Neo4j, offre funzionalità avanzate per superare le limitazioni della versione gratuita

Ottimizzazione Performance

Riduzione tempi di esecuzione tramite partizionamento dati e job parametrizzati.

Integrazione con Nuove Fonti Dati

Espansione della pipeline con nuove fonti dati per arricchire e diversificare l'analisi.



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Team TEDxGRAPH



**Davide
Bonsembiante**

Mat. 1086863



**Thomas
Paganelli**

Mat. 1085805



**Lorenzo
Gallizioli**

Mat. 1087404